# Tending to Troy

## Claude Engineering Guide & Build Roadmap

**Version 1.0**

---

# Purpose

This document explains how Claude should approach the development of Tending to Troy.

The objective is not simply to generate working software.

The objective is to produce software that feels handcrafted, maintainable and enjoyable to use for many years.

Claude should act as a senior software engineer working within an existing product team.

Where uncertainty exists, favour simplicity over feature creep.

---

# Project Philosophy

Tending to Troy is intentionally small.

Do not attempt to compete with commercial task management software.

Every feature should reinforce calmness and simplicity.

Whenever a feature risks adding unnecessary complexity, stop and reconsider.

Ask:

"Does this genuinely improve caring for the home?"

If the answer is no, leave it out.

---

# Design Philosophy

Respect the Design System document.

Never replace specified colours or typography with framework defaults.

Do not substitute Material Design components.

Avoid generic SaaS layouts.

The application should feel premium and personal.

Animations should always remain subtle.

---

# Engineering Standards

Write production-quality code.

Avoid shortcuts.

Avoid placeholder implementations unless requested.

Prefer readability over cleverness.

Keep components small and reusable.

Keep files organised.

Follow strict TypeScript conventions.

Avoid unnecessary dependencies.

---

# Git Workflow

Develop each feature in isolation.

Each feature should represent a logical commit.

Examples:

Authentication

Task creation

Task editing

Priority movement

Weather

Search

Archive

Shopping

Avoid massive commits.

---

# Build Order

The application should be developed in the following sequence.

---

## Phase 1

Project setup

Repository

TypeScript

Next.js

Tailwind

Supabase

Authentication

Deployment

---

## Phase 2

Core layout

Navigation

Home screen

Priority sections

Task cards

Typography

Colour system

Responsive layout

---

## Phase 3

Task system

Create

Edit

Delete

Archive

Restore

Pin

Priority movement

Manual ordering

---

## Phase 4

Task detail

Description

Checklist

Shopping

Links

Notes

History

---

## Phase 5

Search

Search by title

Filtering

Performance optimisation

---

## Phase 6

Weather

Met Office integration

Outdoor banner

Caching

Failure handling

---

## Phase 7

Polish

Animations

Transitions

Loading states

Accessibility

Responsive improvements

Performance optimisation

---

### Phase 8

Testing

Unit tests

Integration tests

Manual QA

Bug fixing

---

# Review Points

Stop after each phase.

Do not immediately continue.

Provide:

Summary

Screenshots

Architecture decisions

Questions

Potential improvements

Await approval before continuing.

---

# User Experience Rules

Never require unnecessary taps.

Every interaction should feel obvious.

Navigation should be effortless.

Users should never wonder where something is.

Avoid hidden functionality.

Avoid complex menus.

Prefer visible options.

---

# Editing

Every edit auto-saves.

Never require a Save button.

If a save fails:

Retry automatically.

Display a gentle message.

Never lose user input.

---

# Deletion

Deletion should always require confirmation.

Accidental deletion should be difficult.

Archived tasks should remain recoverable.

---

# Performance Goals

Opening application

Feels instant.

Opening task

Feels immediate.

Dragging

Perfectly smooth.

Searching

Instant.

Animations

Never delay interaction.

---

# Code Reviews

Before implementing new functionality ask:

Can this be simpler?

Can this be reused?

Does this match the design philosophy?

Would another developer immediately understand this?

---

# Feature Requests

Future features should be implemented only when requested.

Do not anticipate functionality.

Avoid speculative coding.

Build only what exists in the current specification.

---

# Error Messages

Use plain English.

Examples:

Unable to save changes.

Please check your connection.

Task couldn't be archived.

Never expose stack traces.

Never expose technical language.

---

# Accessibility

Keyboard navigation.

Screen reader labels.

Large touch targets.

Appropriate contrast.

Respect user font scaling.

---

# Documentation

Every major feature should include:

Purpose

Architecture

Important decisions

Potential future improvements

The project should remain understandable after several years.

---

# Refactoring

When refactoring:

Do not change behaviour.

Reduce complexity.

Improve naming.

Improve organisation.

Avoid unnecessary rewrites.

---

# Future Architecture

The following features are expected in future versions.

The architecture should allow them without significant redesign.

AI shopping assistant.

Retail price comparison.

Monzo integration.

Budget forecasting.

Photo attachments.

Equipment register.

Recurring maintenance.

Multiple homes.

Widgets.

Manual storage.

House inventory.

Apple orchard management.

Log cabin maintenance.

---

# Things to Avoid

Do not introduce unnecessary dashboards.

Do not gamify the application.

Do not add achievements.

Do not add streaks.

Do not add productivity statistics.

Do not add motivational quotes.

Do not add advertising.

Do not collect analytics unnecessarily.

Do not create unnecessary settings screens.

The application should remain intentionally focused.

---

# Quality Checklist

Before considering Version 1 complete, verify:

Authentication works.

Tasks synchronise correctly.

Priority movement functions.

Pinning works.

Archive works.

Restore works.

Search works.

Weather functions.

Shopping lists work.

Links open correctly.

Auto-save functions.

Responsive layout works.

PWA installs correctly.

Performance feels excellent.

Typography matches the specification.

Colours match the design system.

Animations feel subtle.

Accessibility requirements are met.

No obvious bugs remain.

---

# Definition of Done

Version 1 is complete when:

The application feels calm.

The interface feels handcrafted.

The software disappears into the background.

Users instinctively understand how to use it without needing instructions.

Opening Tending to Troy should feel like opening a trusted household journal rather than launching another app.

If a feature improves clarity, simplicity or care for the home, it belongs.

If it adds noise, distraction or unnecessary complexity, it does not.

The software should age gracefully and remain useful for many years with only incremental improvements.

The guiding principle for every decision is simple:

**Care for our home.**